LECTURE 2

Java Networking Fundamentals

Building connected applications with TCP/IP

Today's Agenda

1. Network Protocols Primer
2. TCP vs UDP: Comparison & Tradeoffs
3. IP Addressing: IPv4 and IPv6
4. Java Networking Classes
5. Network Security Basics
6. Summary & Best Practices

PART 1

Network Protocols Primer

Understanding how computers talk to each other

Network Protocol

A set of rules governing communication between networked devices.

- Specifies message format, sequence, and error handling
- Ensures all devices follow the same 'language'
- Defines ports, packet structure, and acknowledgments
- Example: HTTP (web), FTP (file transfer), SMTP (email)

Open Systems Interconnection (OSI) Model Overview (7 Layers Simplified)

- Layer 1 (Physical): Hardware cables, signals, RF
- Layer 2 (Data Link): MAC addresses, switches, Ethernet
- Layer 3 (Network): IP routing, addresses, packets
- Layer 4 (Transport): TCP/UDP, reliability, ports
- Layer 5-7 (Session/Presentation/Application): HTTP, FTP, SMTP, DNS
- Data flows down through layers on send, up on receive

Transmission Control Protocol (TCP)/IP Stack

Application	HTTP, FTP, SMTP, DNS, SSH, Telnet
Transport	TCP (reliable), UDP (fast), SCTP
Internet	IP (IPv4, IPv6), ICMP, IGP routing
Network Access	Ethernet, WiFi, PPP, hardware drivers

Key Points

- TCP/IP is 4-layer model
- Most used protocol suite
- Connectionless and connection-oriented

TCP/IP Model (Simplified)

1

Application

HTTP, FTP, SMTP, DNS, SSH

2

Transport

TCP (reliable), UDP (fast)

3

Internet

IP (IPv4, IPv6) routing

4

Link

Ethernet, WiFi, hardware

Key Application Layer Protocols

- HTTP/HTTPS: Web browsing (port 80/443)
- FTP: File transfer (port 21)
- SMTP: Email sending (port 25/587)
- POP3/IMAP: Email retrieval (port 110/143)
- DNS: Domain name resolution (port 53)
- SSH: Secure shell (port 22)
- TELNET: Remote login (port 23, insecure)

Java Code Snippets: HTTP, FTP & SMTP (Part 1 of 2)

```
// HTTP/HTTPS (port 80/443) — HttpURLConnection
URL url = new URL("https://example.com");
HttpURLConnection con = (HttpURLConnection) url.openConnection();
con.setRequestMethod("GET");
int status = con.getResponseCode(); // 200, 404, etc.
```

```
// FTP (port 21) — Apache Commons Net
FTPClient ftp = new FTPClient();
ftp.connect("ftp.example.com", 21);
ftp.login("user", "pass");
ftp.retrieveFile("file.txt", new FileOutputStream("file.txt"));
```

```
// SMTP (port 587) — Jakarta Mail
Properties props = new Properties();
props.put("mail.smtp.host", "smtp.gmail.com");
props.put("mail.smtp.port", "587");
Session session = Session.getInstance(props, authenticator);
Transport.send(new MimeMessage(session)); // send email
```

Java Code Snippets: POP3/IMAP, DNS, SSH & Telnet

```
// POP3/IMAP (port 110/143) — Jakarta Mail
Store store = session.getStore("imap"); // or "pop3"
store.connect("imap.gmail.com", "user", "pass");
Folder inbox = store.getFolder("INBOX");
inbox.open(Folder.READ_ONLY);
Message[] msgs = inbox.getMessages(); // fetch emails
```

```
// DNS (port 53) — InetAddress lookup
InetAddress addr = InetAddress.getByName("www.example.com");
System.out.println(addr.getHostAddress()); // prints IP
```

```
// SSH (port 22) — JSch library
JSch jsch = new JSch();
Session s = jsch.getSession("user","host",22); s.connect();
Channel ch = s.openChannel("exec"); ch.connect();
```

```
// TELNET (port 23) — Raw Socket (insecure)
Socket socket = new Socket("host", 23);
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
out.println("command"); // Prefer SSH — Telnet is plaintext!
```

HTTP Request-Response Cycle

How web requests travel across the network

Client

Browser sends request

→

DNS Lookup

Resolve hostname

→

Server

Receives request

→

Process

Server executes logic

→

Response

Client receives data

HTTP Request/Response Details

- **1. Client sends an HTTP request to the server**
  - **Client** — the one asking (e.g. your browser or a Java app)
  - **HTTP Method** — the intent: GET (fetch), POST (create), PUT (update), DELETE (remove)
  - **Think of it as** placing an order at a restaurant — you tell the waiter what you want
- **2. Every request is made up of 4 parts:**
  - **Method** — what action to perform (e.g. GET = read, POST = create)
  - **Path** — which resource to access (e.g. /api/users/42)
  - **Headers** — extra info/metadata (e.g. Content-Type: application/json)
  - **Body** — optional data payload (e.g. JSON sent with POST/PUT; absent in GET)

HTTP Request/Response Details (cont.)

- **3. Server receives, processes, and sends back a response**
  - **Receives** — reads the request and understands what is being asked
  - **Processes** — runs logic, queries a database, or performs the needed action
  - **Responds** — sends back an HTTP response with the result
- **4. Response also has 3 parts:**
  - **Status Code** — did it work? (200 = OK, 404 = not found, 500 = server crashed)
  - **Headers** — metadata about the response (e.g. Content-Type: text/html)
  - **Body** — the actual content returned (e.g. an HTML page or JSON data)
  - *Code families: 1xx (info), 2xx (success), 3xx (redirect), 4xx (client error), 5xx (server error)*
- **5. Connection closes after the exchange**
  - **HTTP/1.0** — closes the connection after every single request (slow)
  - **HTTP/1.1 Keep-Alive** — keeps the connection open to reuse for multiple requests (faster)
  - **Analogy** — like leaving the restaurant vs. staying at your table to order again

Network Protocols Overview

HTTP/HTTPS

HyperText Transfer Protocol. Web pages. Port 80/443. Stateless.

FTP

File Transfer Protocol. Move files. Port 21. Unencrypted (SFTP is encrypted).

SMTP

Simple Mail Transfer Protocol. Send email. Port 25/587. SMTP auth for credentials.

SSL/TLS

Secure Sockets Layer / Transport Layer Security. Encrypts data. HTTPS uses TLS.

PART 2

TCP vs UDP: Comparison & Tradeoffs

Transport layer protocols

TCP (Transmission Control Protocol)

Connection-oriented, reliable protocol that ensures data delivery.

- Establishes connection before communication (handshake)
- Guarantees data delivery in order
- Automatic retransmission of lost packets
- Flow control and congestion control mechanisms

UDP (User Datagram Protocol)

Connectionless, unreliable protocol optimized for speed.

- No connection establishment overhead
- No guaranteed delivery (packets may be lost)
- No ordering guarantees
- Lower latency, suitable for real-time applications

TCP vs UDP Detailed Comparison

TCP

- Connection-oriented
- Guaranteed delivery
- Ordered delivery
- Flow control
- Congestion control
- Higher overhead
- Use: Web, Email, FTP, SSH
- Port: Server-specific

UDP

- Connectionless
- Best-effort delivery
- No ordering guarantee
- No flow control
- No congestion control
- Lower overhead
- Use: DNS, VoIP, Gaming, Streaming
- Port: Broadcast capable

TCP Connection: Three-Way Handshake

- Before two computers send data, they must **agree to connect first** — this is called the Three-Way Handshake
- *Analogy: like a phone call — you don't speak until you know the other person picked up*
- **Step 1 — SYN** (Synchronise): Client says *"Hello, are you there? I want to connect."*
  - Client picks a random starting number (sequence number) so both sides can track packets in order
- **Step 2 — SYN-ACK** (Synchronise + Acknowledge): Server replies *"Yes! Got your message — here's my number."*
  - Server confirms it received the client's SYN and sends its own sequence number back
- **Step 3 — ACK** (Acknowledge): Client confirms *"Got it — let's talk!"*
  - Both sides now know the other is ready — connection established, data can flow
- **Why does this matter?**
  - Guarantees both sides are alive and ready — no data is wasted on a dead connection
  - Cost: adds ~1 round-trip delay before the first byte of real data is sent
  - *This is why TCP is reliable but slightly slower than UDP, which skips the handshake entirely*

When to Use TCP vs UDP

- Use TCP: Email, file transfer, web browsing (data integrity critical)
- Use UDP: Live video streaming, VoIP, online gaming (speed critical)
- Use TCP: Banking, shopping, any financial transaction
- Use UDP: DNS queries (fast, can be retried), NTP time sync
- Use TCP: SSH, any interactive remote command
- Use UDP: Multiplayer games, real-time multimedia

PART 3

IP Addressing: IPv4 and IPv6

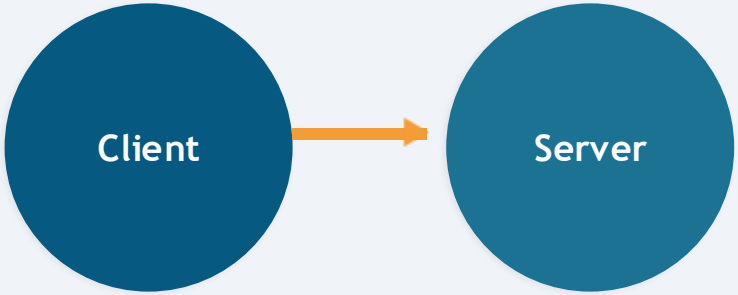
Internet Protocol addresses

IP Address

A numerical label assigned to each device connected to a network.

- IPv4: 32-bit address (4 octets), e.g., 192.168.1.1
- IPv6: 128-bit address (8 groups of 4 hex), e.g., 2001:0db8::1
- Enables routing of data packets across networks
- Unique identification in network communication

IP Address Concept



- IP addresses identify machines on a network
- IPv4: 32-bit (4.3 billion possible addresses) - almost exhausted
- IPv6: 128-bit (340 undecillion addresses) - future-proof
- Loopback address 127.0.0.1 (IPv4) or ::1 (IPv6) for local testing
- Private ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16

IPv4 vs IPv6: Limitations vs Capabilities

IPv4 Limitations

- 32-bit address space (4.3 billion addresses)
- Address exhaustion already happening
- NAT (Network Address Translation) workaround
- No built-in quality of service (QoS)
- Fragmentation at intermediate routers

IPv6 Capabilities

- 128-bit address space (340 trillion trillion)
- Essentially unlimited addresses
- No NAT needed; every device gets unique address
- Built-in QoS for quality of service
- Simplified header processing and routing

IPv4 Address Classes & Private Ranges

- An IPv4 address is **4 numbers separated by dots** (e.g. 192.168.1.5), each 0–255
- *Analogy: like a postal address — the first part identifies the city, the rest the street and house*
- **Address Classes —** originally used to define how big a network could be:
  - **Class A** (1.x.x.x – 126.x.x.x) — huge networks, up to ~16 million hosts each (e.g. a global ISP)
  - **Class B** (128.x.x.x – 191.x.x.x) — medium networks, up to ~65,000 hosts (e.g. a university)
  - **Class C** (192.x.x.x – 223.x.x.x) — small networks, up to 254 hosts (e.g. a home or office)
- **Private addresses —** reserved ranges that only work **inside** a local network, never routed on the internet:
  - 10.x.x.x — Class A private (large internal networks)
  - 172.16.x.x – 172.31.x.x — Class B private
  - 192.168.x.x — Class C private (your home Wi-Fi router uses this!)

IPv4 Subnetting & CIDR Notation

- **Subnetting** = dividing one large network into smaller, manageable sub-networks
- *Analogy: a building has one street address, but many apartments inside — subnets are the apartments*
- **Subnet Mask —** tells the computer which part of the IP is the **network** vs. the **device**
  - Example: IP **192.168.1**.5 with mask 255.255.255.0 → first 3 numbers = network, last = device
  - 255 means "this part identifies the network"; 0 means "this part identifies the host"
- **CIDR Notation —** a shorthand that replaces the long subnet mask with a single number
  - **192.168.1.0/24** means: first 24 bits = network → same as mask 255.255.255.0
  - The remaining 8 bits are for devices → 2^8 = 256 addresses (254 usable; 1 network + 1 broadcast reserved)
  - *Bigger /number = smaller network: /24 has 254 hosts, /16 has ~65,000, /8 has ~16 million*

Ports & Socket Communication

- Port: 16-bit number (0-65535) identifying service on host
- Well-known ports: 0-1023 (HTTP=80, HTTPS=443, SSH=22, FTP=21)
- Registered ports: 1024-49151 (application-specific)
- Dynamic/ephemeral ports: 49152-65535 (temporary connections)
- Socket = IP address + port number (e.g., 192.168.1.100:8080)
- Multiple processes can listen on different ports same IP

PART 4

Java Networking Classes

java.net and java.nio packages

Core Java Networking Classes (java.net)

- InetAddress: Represents IP address (IPv4 or IPv6)
- Socket: Client-side TCP connection
- ServerSocket: Server-side TCP socket (listens for connections)
- DatagramSocket: UDP socket for connectionless communication
- URL: Represents web URL with parsing and resource access
- URLConnection: Opens connection to URL and reads/writes data

Creating a TCP Client Socket

JAVA

```java
try {
    Socket socket = new Socket("example.com", 80);
    InputStream in = socket.getInputStream();
    OutputStream out = socket.getOutputStream();

    out.write("GET / HTTP/1.1\r\n".getBytes());
    out.write("Host: example.com\r\n\r\n".getBytes());
    out.flush();

    BufferedReader reader = new BufferedReader(
        new InputStreamReader(in));
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
    socket.close();
} catch (IOException e) {
    e.printStackTrace();
}
```

Low-level TCP. Use HttpClient or libraries for HTTP in practice

Creating a TCP Server Socket

JAVA

```java
try {
    ServerSocket serverSocket = new ServerSocket(8080);
    System.out.println("Server listening on port 8080");

    while (true) {
        Socket clientSocket = serverSocket.accept();
        System.out.println("Client connected: " +
            clientSocket.getInetAddress());

        OutputStream out = clientSocket.getOutputStream();
        out.write("Hello Client!".getBytes());
        out.flush();
        clientSocket.close();
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

Single-threaded server. Use thread pool for multiple clients

InetAddress: Hostname Resolution

JAVA

```java
try {
    // Resolve hostname to IP address
    InetAddress addr = InetAddress.getByName("google.com");
    System.out.println("Hostname: " + addr.getHostName());
    System.out.println("IP: " + addr.getHostAddress());

    // Get local machine address
    InetAddress local = InetAddress.getLocalHost();
    System.out.println("Local IP: " + local.getHostAddress());

    // Check if address is reachable
    if (addr.isReachable(2000)) {
        System.out.println("Host is reachable");
    }
} catch (UnknownHostException e) {
    System.out.println("Host not found");
}
```

isReachable() sends ICMP echo request or TCP probe

URL & URLConnection

- URL: Uniform Resource Locator (web address)
- URLConnection: Opens connection to remote resource
- Can set headers, timeout, read response
- Suitable for HTTP requests (though HttpClient preferred)
- Can read from any URL: http, https, file, ftp
- Use for simple requests; frameworks for complex applications

URL Connection Example

JAVA

```java
try {
    URL url = new URL("https://api.example.com/data");
    URLConnection conn = url.openConnection();

    // Set request properties
    conn.setRequestProperty("User-Agent", "Java-Client");
    conn.setConnectTimeout(5000);
    conn.setReadTimeout(5000);

    // Read response
    BufferedReader in = new BufferedReader(
        new InputStreamReader(conn.getInputStream()));
    String line;
    while ((line = in.readLine()) != null) {
        System.out.println(line);
    }
    in.close();
} catch (IOException e) {
    e.printStackTrace();
}
```

For JSON APIs, use libraries like Jackson or Gson for parsing

End-to-End Network Communication

SCENARIO

You type 'www.google.com' in your browser and press Enter. Describe the complete journey of your request from your keyboard to Google's server and back to your screen.

THINK ABOUT

1. What role does DNS play in this process?

2. How many different protocols are involved?

3. What happens at your router and ISP?

PART 5

Network Security Basics

Encryption and secure communication

SSL/TLS (Secure Sockets Layer / Transport Layer Security)

Cryptographic protocol providing secure communication over network.

- Encrypts data in transit (confidentiality)
- Authenticates server (and optionally client)
- Ensures data integrity (detection of tampering)
- TLS 1.3 is current standard; SSL is deprecated

SSL/TLS Handshake Process

Establishing secure connection

ClientHello

Client initiates, sends supported ciphers

→

ServerHello

Server selects cipher, sends certificate

→

Certificate

Server proves identity with cert

→

KeyExchange

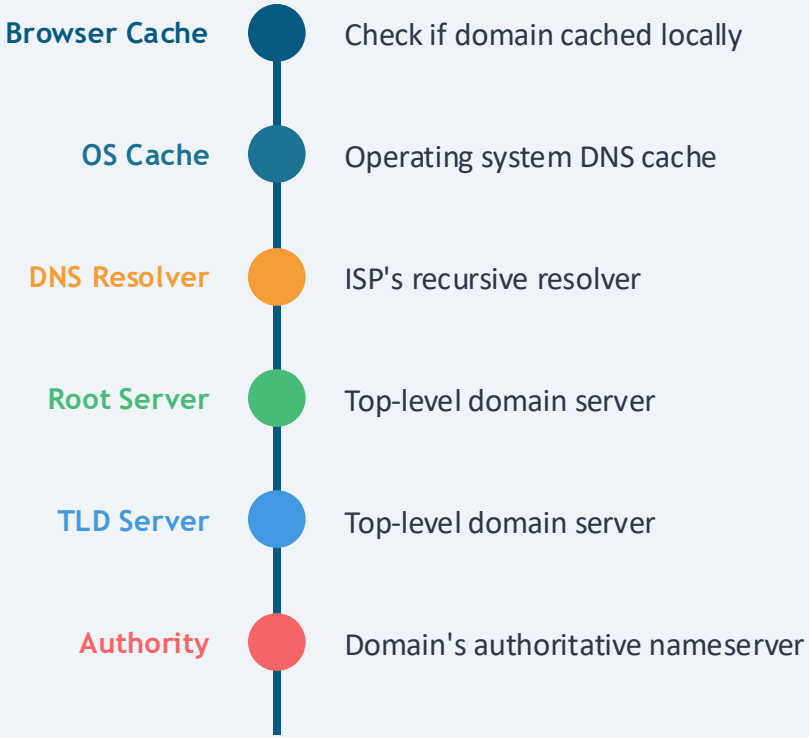
Parties establish shared secret

→

Encrypted

All data encrypted with shared key

DNS Resolution Process



HTTPS and Certificate Authentication

- HTTPS = HTTP + TLS encryption layer
- Server provides digital certificate (issued by CA)
- Certificate contains public key and server identity
- Browser verifies certificate chain (trusted CAs)
- If certificate invalid or expired, browser shows warning
- Self-signed certificates common in development/testing

Java HTTPS / Secure Connections

- java.net.HttpsURLConnection: Extends URLConnection with SSL/TLS
- Automatically handles certificate validation
- Can disable certificate verification (NOT recommended for production)
- System properties: javax.net.ssl.trustStore, keyStore
- Keytool utility manages certificates and keystores
- Most modern frameworks (Spring, Jersey) handle HTTPS seamlessly

HTTPS Connection with Java

JAVA

```java
try {
    URL url = new URL("https://api.example.com/data");
    HttpsURLConnection conn =
        (HttpsURLConnection) url.openConnection();

    // Certificate validation happens automatically
    conn.setConnectTimeout(5000);
    conn.setReadTimeout(5000);

    int responseCode = conn.getResponseCode();
    System.out.println("Response: " + responseCode);

    BufferedReader in = new BufferedReader(
        new InputStreamReader(conn.getInputStream()));
    String line;
    while ((line = in.readLine()) != null) {
        System.out.println(line);
    }
    in.close();
} catch (Exception e) {
    e.printStackTrace();
}
```

Modern Java validates certificates by default. Trust manager can customize

Think — Pair — Share

Network troubleshooting requires understanding of multiple layers

A company's internal website works fine on the local network but external users cannot access it. What are all the possible reasons and how would you troubleshoot?

1 THINK

Reflect individually for 1 minute

2 PAIR

Discuss with your neighbor for 2 minutes

3 SHARE

Share insights with the class

Hints: Firewall | NAT | DNS | Ports | Routing

PART 6

Summary & Best Practices

Key takeaways on networking

Key Networking Concepts

1. Networks use protocols (TCP/IP) for standardized communication
2. TCP for reliability (web, email); UDP for speed (gaming, streaming)
3. IP addresses identify machines; ports identify services
4. IPv4 exhausted; IPv6 provides virtually unlimited addresses
5. DNS resolves hostnames to IP addresses
6. HTTPS uses TLS for encryption and server authentication

Java Networking Best Practices

- Use HttpClient (Java 11+) or OkHttp/Retrofit for HTTP requests
- Always set connection and read timeouts (prevents hanging)
- Use HTTPS for any sensitive data (authentication, payments)
- Validate SSL certificates (never disable in production)
- Implement retry logic for transient network failures
- Monitor network latency and connection pools in production

Common Networking Pitfalls

- Hardcoding IP addresses instead of using DNS names
- Not handling timeouts (connections hang indefinitely)
- Ignoring connection pooling (creates too many TCP connections)
- Disabling certificate validation for convenience (SECURITY RISK)
- Not implementing proper error handling and retries
- Blocking main thread on network I/O (UI freezes)

Questions?

Next: Lecture 3 — Java Threads & Concurrency

CSCI 2450 | Computer Interface Programming | 2026